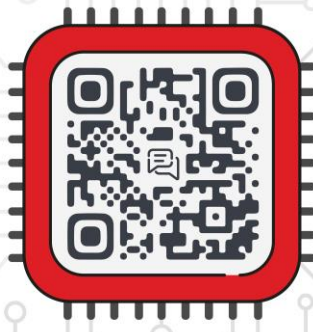




3



5



7.5 sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

التوابع في جافا

د. ماهر صارم



RB Informatcs; 16/4/2026

البرمجة 2

تعدد الأشكال (Polymorphism)

وهو المفهوم الذي يسمح للدوال أو التوابع بأن تأخذ أكثر من شكل، وله نوعان:

1- التحميل الزائد (Over loading):

التحميل الزائد هو عبارة عن كتابة تابعين أو أكثر بنفس الاسم ولكن يختلف نوع الوسيط فيه أو عدد الوسائط (Parameter).

2- التجاوز (Over riding):

هو عندما يكون لدينا تابع موجود في صف معين (Class) ونقوم بإعادة تعريف هذا التابع بحيث يقوم بوظيفة أخرى.

ملاحظة: سنتعرف أكثر على التجاوز في المحاضرات القادمة.

People who avoid failure
also avoid success.



تمرين عن فكرة التحميل الزائد:

تابع إيجاد أكبر قيمة بين عددين وتابع إيجاد أكبر قيمة بين 3 أعداد:

```
1- public class Main {
2-     public static void main(String[] args) {
3-         int num1=10;
4-         int num2=5;
5-         int num3=17;
6-         System.out.println(maxNumber(num1,num2));
7-         System.out.println(maxNumber(num1,num2,num3));
8-     }
9-     public static int maxNumber(int num1,int num2)
10-    {
11-        if(num1>num2)
12-            return num1;
13-        else return num2;
14-    }
15-     public static int maxNumber(int num1,int num2,int num3)
16-    {
17-        int m=maxNumber(num1,num2);
18-        if(num3>m)
19-            return num3;
20-        else return m;
21-    }
22- }
```

شرح البرنامج السابق:

9 <= 13: تابع يحوي وسيطين يتحقق إذا كان العدد الأول أكبر من الثاني، يعيد قيمة الأول وإلا يعيد قيمة الثاني.

14 <= 18: تابع آخر نفس اسم التابع السابق ولكن من أجل 3 أعداد وهنا تمت عملية التحميل الزائد حيث قمنا بوضع 3 وسائط بدل وسيطين.

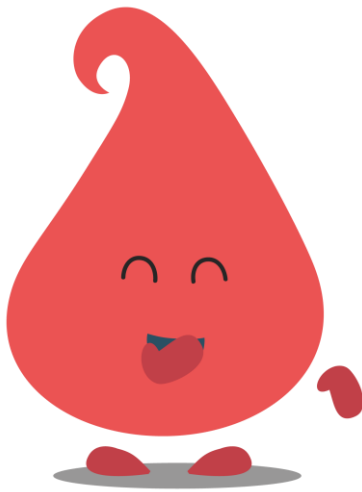
نلاحظ بالسطر 17 بدل من إعادة كتابة سطر الكود للمقارنة بين أول عددين قمنا باستدعاء التابع السابق من أجل العددين num1 و num2 و خزنا القيمة العائدة منه في متغير m ثم قارنا بين m والعدد الثالث وأعدنا قيمة الأكبر بعد المقارنة.

العودية (Recursion)

العودية هي عملية استدعاء التابع لنفسه أكثر من مرة حتى يصل لشروط توقف معين.

ماذا لو لم نضع شرط للتوقف؟

إذا لم نضع شرط للتوقف، سيستمر التابع باستدعاء نفسه مراراً وتكراراً إلى مالا نهاية، وبما أن كل استدعاء يحتاج لمساحة في الذاكرة (Stack) لتخزين بيانات هذا الاستدعاء ستمتلئ الذاكرة المخصصة لهذا البرنامج ويحدث خطأ يسمى: Stack overflow.



تمرين على فكرة العودية:

العامل (factorial):

شرح البرنامج:

```
1- public class Main {  
2-     public static void main(String[]  
args) {  
3-         System.out.println(factorial(5));  
4-     }  
5-     public static int factorial(int n){  
6-         if(n==1 || n==0)  
7-             return 1;  
8-         else  
9-             return n*factorial(n-1);  
10-    }  
11- }
```

5: تابع يعيد قيمة int اسمه factorial يمر عبره وسيط من نوع int وهو التابع الذي سيقوم بعملية حساب العامل.

6: وهو شرط التوقف، عند الوصول لاستدعاء تكون فيه $n = 1$ أو $n = 0$ يقوم التابع بإرجاع قيمة 1.

9: عملية العودية حيث أن التابع سيأخذ قيمة n المدخلة ويضربها بالقيمة المعادة من الاستدعاء $n - 1$.

من أجل $n = 5$

سيقوم التابع بأخذ قيمة n ويضربها بالقيمة العائدة من استدعاء التابع من أجل $n - 1$

أي

$$5 \times factorial(4);$$

$$5 \times 4 \times factorial(3);$$

وسيكبر نفسه حتى يصل إلى

$$factorial(1)$$

عندما $n - 1$ شرط التوقف محقق سيعيد قيمة 1 وتصبح العملية كالآتي

$$5 \times 4 \times 3 \times 2 \times 1$$

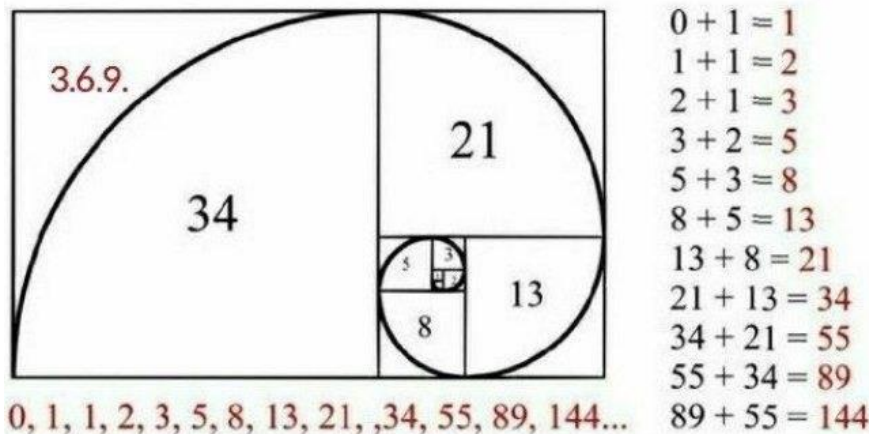
🔗 سلسلة فيبوناتشي (Fibonacci):

هي سلسلة تبدأ من الصفر والواحد، ثم كل رقم تال فيها هو مجموع الرقمين السابقين له.

للإثراء (ليس للحفظ):

هل تعلم؟

أن ناتج قسمة أي رقم في السلسلة على الرقم الذي قبله يقترب دائماً من الرقم 1,618، وهي النسبة التي يعتبرها المصممون والمهندسون سر التوازن والجمال في التصميمات المعمارية.



تمرين: تطبيق سلسلة فيبوناتشي بالعودية:

شرح البرنامج:

```

1- public class Main {
2-     public static void main(String[]
args) {
3-         System.out.println(fib(7));
4-     }
5-     public static int fib(int n)
6-     {
7-         if(n==1||n==0)
8-             return n;
9-         else
10-            return fib(n-1)+fib(n-2);
11-     }
12- }

```

5: شرط التوقف، سلسلة فيبوناتشي تبدأ بالرقمين 0 و 1، لذا في حال كانت $n = 0$ أو $n = 1$ عندها لا حاجة لحساب شيء، فقط نعيد قيمة n .

8: في سلسلة فيبوناتشي كل حد يساوي مجموع الحدين السابقين له وهنا التابع يستدعي نفسه من أجل القيمتين السابقتين $n - 1$ و $n - 2$ وجمع القيمتين العائدتين معاً ليعطي قيمة n .

مثال توضيحي:

نفرض $n = 4$.

التابع يرى $fib(n)$ فيقسمها لـ $fib(2) + fib(3)$

يذهب ليحسب $fib(3)$ فيقسمها لـ $fib(1) + fib(2)$

يستمر في النزول حتى يصل لاستدعاء $fib(1)$ أو $fib(0)$ وهنا يتوقف ويعيد القيم 0 و 1.

تبدأ النتائج بالصعود بالجمع من الأسفل إلى الأعلى.



انتهت المحاضرة الثالثة، بالتوفيق 🍀